



EduConnect Frontend, Backend & Viva Questions

Detailed viva notes for explaining frontend-backend connection, your role, teammate role, challenges, workflows, problem statement, and common project questions with EduConnect examples.

[Frontend](#)[Backend](#)[Database](#)[Roles](#)[Challenges](#)[Viva Q&A](#)

Prepared for EduConnect major project viva.

1. Problem Statement

University students often face difficulty finding syllabus-based academic resources in one place. Notes, videos, PYQs, quizzes, and live class links are usually scattered across WhatsApp groups, Telegram channels, random PDFs, and YouTube playlists. This creates confusion and wastes students' time.

Teachers also need a structured platform to upload subject-wise content, and admins need a proper system to manage academic hierarchy, teachers, subscriptions, approvals, and student access.

Problem Statement: To develop a role-based ERP + LMS platform that provides centralized, syllabus-based academic resources and manages students, teachers, content, quizzes, subscriptions, and academic hierarchy in a structured way.

2. Proposed Solution

EduConnect solves this problem by organizing learning resources through academic hierarchy:

University -> Course -> Branch -> Scheme -> Semester -> Subject -> Unit -> Topic -> Notes / Videos / PYQs / Quizzes / Live Classes

The system has four roles: student, teacher, admin, and maintenance. Each role has different access and responsibilities.

3. Frontend To Backend Connection

The frontend is developed using React and TypeScript. The backend is developed using Node.js. The database is MySQL. React components do not directly connect to MySQL. Instead, they use a client wrapper.

React Component -> `src/integrations/mysql/client.ts` -> `server/mysql-api.mjs` -> MySQL Database -> API Response -> React UI Update

This architecture separates the UI from database logic. It improves maintainability because frontend components only call reusable client methods such as select, insert, update, and delete.

Example Frontend Call

```
const { data, error } = await mysqlClient
  .from("subjects")
  .select("*")
  .eq("semester", 1);
```

This code sends a request to backend. The backend fetches matching subjects from MySQL and sends the result back to React.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React, TypeScript, Vite	Dynamic UI, reusable components, fast development.
Styling	Tailwind CSS, shadcn/Radix UI	Responsive and modern user interface.
Routing	React Router DOM	Page navigation and protected routes.
Data Fetching	TanStack React Query	API data fetching and caching.
Backend	Node.js	API layer between frontend and database.
Database	MySQL	Stores users, roles, content, quizzes, subscriptions and hierarchy.
Icons/UI	Lucide React, Framer Motion	Icons and smooth UI animations.

5. Roles And Responsibilities

Role	Responsibilities	EduConnect Example
Student	Access learning resources, quizzes, PYQs, subscriptions, live classes and teacher chat.	Student opens syllabus, attempts quiz, downloads notes.
Teacher	Upload notes, videos, PYQs, create quizzes and schedule live classes for assigned subjects.	Teacher uploads DBMS notes only if admin assigned DBMS.
Admin	Manage hierarchy, approve teachers, assign subjects, approve content, verify payments.	Admin assigns MCA 201 DBMS to a teacher.
Maintenance	Raise academic hierarchy update requests for admin approval.	Maintenance requests new 2025 scheme addition.

6. Your Role And Teammate Role

Your Role • Frontend pages and components. • Backend/API integration. • Role-based dashboards. • Quiz flow, timer, answer saving, leaderboard. • Subscription and payment screenshot workflow. • Teacher upload and admin approval UI integration.	Teammate Role • MySQL database design. • Table creation and relationships. • Data insertion and seed data. • Maintaining syllabus, quiz, and academic records.
---	---

7. Challenges Faced And Solutions

Challenge	Problem	Solution
Frontend-backend integration	React could not directly use MySQL.	Used Node.js API and mysqlClient wrapper.
Role-based access	Different users needed different permissions.	Used user_roles table and RoleRoute.tsx.
Quiz answer saving	Answers could duplicate or submit incorrectly.	Saved final answers on submit into student_answers.
Quiz timer	Timer needed to work per question.	Implemented 20 seconds per question with auto-next and auto-submit.
Subscription access	Free and premium access needed separation.	Used semester-wise subscriptions and admin approval.
Teacher restrictions	Teachers should not upload for all subjects.	Used teacher_assignments table for subject-wise access.

8. Important Database Tables

Table	Purpose
app_users	Stores login email and password.
profiles	Stores profile details such as name, phone, gender, semester.
user_roles	Stores student, teacher, admin, maintenance roles.
universities, courses, branches, schemes, subjects, units, syllabus_topics	Stores academic hierarchy.
content	Stores notes, videos, PDFs, topic mapping and approval status.
teacher_assignments	Stores teacher-subject mapping.
quizzes, questions, options	Stores quiz structure.
quiz_attempts, student_answers	Stores quiz result and each selected answer.
subscriptions, payments	Stores premium request and payment proof.
hierarchy_requests	Stores maintenance requests.

9. Common Viva Questions With Detailed Answers

1. How did you connect frontend to backend?

In EduConnect, the frontend is built using React and TypeScript, and the backend is built using Node.js with MySQL. The React components do not directly connect to MySQL. Instead, they use a frontend API client wrapper named `src/integrations/mysql/client.ts`. This wrapper sends requests to the Node.js backend file `server/mysql-api.mjs`. The backend performs the required

MySQL operation and returns the response to the frontend. This keeps frontend and database separated and makes the project easier to maintain.

EduConnect Example: Example flow: React component -> mysqlClient wrapper -> Node.js backend API -> MySQL database -> API response -> React UI update.

2. What was your role in the project?

My role was mainly frontend development and backend/API integration. I worked on React pages, role-based dashboards, syllabus navigation, quiz flow, subscription request workflow, teacher upload workflow, admin approval screens, and connecting frontend components with backend API data.

EduConnect Example: For example, I worked on quiz UI, quiz timer, answer submission, subscription page, dashboards, and role-based routing.

3. What was your teammate's role?

My teammate mainly worked on the database part. This includes MySQL table design, creating relationships, inserting initial data, and maintaining database structure for users, roles, academic hierarchy, quizzes, payments, and content.

EduConnect Example: For example, database tables like subjects, units, questions, options, subscriptions, and student_answers are part of the database layer.

4. Why did you use React?

React is useful for building dynamic and reusable UI components. EduConnect has multiple interactive screens such as dashboards, syllabus navigation, quiz timer, subscription form, admin panels, and teacher upload screens. React makes it easier to split these into reusable components.

EduConnect Example: For example, QuizBrowser displays quiz cards, TakeQuiz handles quiz attempt, and CoursePlayer handles notes/video player.

5. Why did you use Node.js backend?

Node.js works well as a lightweight API layer between React and MySQL. It handles requests from frontend, runs database queries, manages upload paths, and sends JSON responses back to React.

EduConnect Example: In EduConnect, server/mysql-api.mjs handles backend API operations.

6. Why did you use MySQL?

MySQL is suitable because EduConnect has structured relational data. Users, roles, universities, courses, subjects, units, teachers, quizzes, questions, subscriptions, and payments

are related to each other. A relational database is appropriate for this type of project.

EduConnect Example: For example, one subject has many units, one quiz has many questions, and one student quiz attempt has many student answers.

7. What is role-based access control in EduConnect?

Role-based access control means each user can access only the features allowed for their role. EduConnect has student, teacher, admin, and maintenance roles. The frontend protects routes using `RoleRoute.tsx`, and roles are stored in the `user_roles` table.

EduConnect Example: Example: student cannot access admin teacher assignment page; teacher can upload content only for assigned subjects; admin can approve subscriptions.

8. How does quiz submission work?

When a student starts a quiz, the frontend loads quiz questions and options. The student selects answers. On submit or auto-submit, the system calculates score, creates/updates a `quiz_attempts` record, and saves each selected answer in `student_answers`.

EduConnect Example: Tables used: quizzes, questions, options, `quiz_attempts`, `student_answers`.

9. How does the quiz timer work?

The quiz timer is implemented in the `TakeQuiz` component. Each question gets 20 seconds. If time becomes zero, the system automatically moves to the next question. If it is the last question, the quiz auto-submits.

EduConnect Example: This is an example of application-level algorithm logic used in EduConnect.

10. How does subscription work?

Student selects a semester premium plan, scans QR, pays, uploads payment screenshot and transaction ID. The system creates subscription and payment records with pending status. Admin verifies the screenshot and activates premium access.

EduConnect Example: Tables used: subscriptions and payments.

11. How does teacher assignment work?

Admin approves teacher access and assigns teachers to subjects. Teacher can only manage assigned subjects. This prevents teachers from uploading content for unrelated subjects.

EduConnect Example: Table used: `teacher_assignments`.

12. What is the maintenance role?

Maintenance users help maintain academic structure but cannot directly publish critical changes. They raise requests for hierarchy changes, and admin reviews, approves, or rejects them.

EduConnect Example: Table used: hierarchy_requests.

13. What challenges did you face?

Major challenges included frontend-backend integration, role-based access, quiz answer saving, timer logic, subscription proof workflow, and ensuring teachers only access assigned subjects.

EduConnect Example: For example, quiz answers were initially saving incorrectly, so the final logic saves answers cleanly on submit in student_answers.

14. What is the most important feature?

The most important feature is syllabus-based structured learning with role-based control. EduConnect organizes resources according to the university academic hierarchy instead of showing random content.

EduConnect Example: Hierarchy: University -> Course -> Branch -> Scheme -> Semester -> Subject -> Unit -> Topic.

15. What future improvements can be added?

Future improvements include JWT authentication, online payment gateway, cloud file storage, notification system, AI-based doubt solving, better analytics, and mobile app support.

EduConnect Example: These improvements would make the project more scalable and production-ready.

10. Important Algorithms / Logic

Role Checking Algorithm

```
Get logged-in user
Fetch role from user_roles table
If role is admin -> show admin dashboard
If role is teacher -> show teacher dashboard
If role is student -> show student dashboard
If role is maintenance -> show maintenance dashboard
```

Quiz Scoring Algorithm

```
For each question:
  Get selected option
  Find correct option
  If selected option equals correct option:
    Add marks
```

```
    Save answer as correct
Else:
    Save answer as wrong
Store final score in quiz_attempts
Store each answer in student_answers
```

Subscription Access Algorithm

```
If student has active premium subscription for semester:
    Unlock full content
Else:
    Show free preview content only
```

Leaderboard Algorithm

```
Fetch completed quiz attempts
Group attempts by student
Calculate percentage score
Sort by highest percentage
Display rank
```

11. Best Final Viva Explanation

EduConnect is a role-based ERP + LMS platform developed using React, TypeScript, Node.js, and MySQL. It solves the problem of scattered university learning resources by organizing notes, videos, PYQs, quizzes, and live classes according to academic syllabus hierarchy. The project has four roles: student, teacher, admin, and maintenance. Students access structured learning, teachers upload assigned-subject content, admins manage hierarchy and approvals, and maintenance users raise academic update requests. I worked on frontend and backend integration, while my teammate worked on database design and data insertion.